

ДЗ-1: дедлайн 27.09, вс, 23:59

Приём со снижением балла — до 04.10

ПР-1 — сегодня в 14:40

10 минут, на бумаге.

Темы 01–02 и первый семинар.

Сегодняшние лекции не спрашиваются.

Сдать ДЗ-1: ветка, коммит, pull request

```
# стартёр уже лежит в main
$ git switch -c feature/hw1
# поправили строку Автор в README.md
$ git add README.md
$ git commit -m "Заполнить автора в README"
$ git push -u origin feature/hw1
```



Памятка со всеми шагами и типичными ошибками:
справка / ветка, коммит, pull request

Git сам печатает, куда идти дальше:

```
remote: Create a pull request for 'feature/hw1' on GitHub by visiting:
remote:      https://github.com/tikhomirovd/prime-monitor-demo/pull/new/feature/hw1
```

.../pull/new/... — это ещё не pull request, а форма его создания. В бота — адрес вида .../pull/N.

1. **Create pull request** — кнопкой на GitHub. **Merge не нажимать**
2. В бота: /submit и ссылка на pull request
3. Приватный репозиторий: Settings → Collaborators → tikhomirovd. **Иначе ссылка — 404, и работа не сдана**

Программирование на Python для бизнес-аналитики
Магистратура «Бизнес-аналитика и цифровые решения в финансах» · ВШБ НИУ ВШЭ

Тихие ошибки: pandas, типы и подготовка данных

Дмитрий Тихомиров · 19 сентября 2026

Сегодняшние ошибки не падают.
Их видно только по числу,
которое вы **знали заранее**.

деньги приехали не тем типом
группировка потеряла 10 000 клиентов
запись в срез не дошла до кадра
и отдельно — цена: память

На доске: **880 737 565** ₽

Модель данных

из чего сделан кадр

Series, DataFrame и Index

columns — тоже Index: метки колонок

index метки строк	payment_id	paid_at	amount	status
	9bdbaaba...	2025-06-16 17:45:22+00:00	199.0	success
	59dc9120...	2026-07-27 09:12:51+00:00	249.0	success
	...	...	...	...
3147408	...	...	...	...

DataFrame: несколько Series
с общим индексом

Series amount:
index + значения
одного типа: float64

- У каждой колонки **один тип**. У кадра — у каждой колонки свой: `payments.dtypes`
- Индекс — не номер строки, а *метка*. Из-за этого он и мешает, и помогает

Строки — настоящие первые строки выгрузки `payments`, `uuid` обрезаны; всего строк 3 147 409.

Индекс — не номер строки

```
p = pd.DataFrame({
    "status": ["success", "failed",
               "success", "failed",
               "success"],
    "amount": [399, 199, 449, 249, 399]})
failed = p[p["status"] == "failed"]
```

p			failed		
0	success	399			
1	failed	199	→ 1	failed	199
2	success	449			
3	failed	249	→ 3	failed	249
4	success	399			

После фильтра метки **остались старыми**: 1 и 3, а не 0 и 1. Метки 0 в failed нет вовсе.

failed.loc[3, "amount"]	249 — по метке
failed["amount"].iloc[0]	199 — по позиции
failed.loc[0]	KeyError

- .loc — по меткам, .iloc — по позициям
- Нужна нумерация с нуля —
reset_index(drop=True)
- Индекс участвует и в арифметике —
следующий слайд

Индекс складывает за вас

```
>>> a
week
2026-07-06    6765579.0
2026-07-13    6874641.0
2026-07-20    6794214.0
2026-07-27    6857709.0
Name: basic, dtype: float64
```

```
>>> b
week
2026-07-20    7563405.0
2026-07-27    7534669.0
2026-08-03    7650062.0
2026-08-10    7597529.0
Name: extended, dtype: float64
```

```
>>> a + b
week
2026-07-06      NaN
2026-07-13      NaN
2026-07-20    14357619.0
2026-07-27    14392378.0
2026-08-03      NaN
2026-08-10      NaN
dtype: float64
```

`(a + b).sum()` **28 749 997**

`a.sum() + b.sum()` **57 637 808**

Сложение идёт **по меткам**, а не по порядку. Недели, которые есть только в одной Series, дают NaN — 4 из 6, и `.sum()` пропускает их молча. Нужен ноль вместо пропуска — `a.add(b, fill_value=0)`.

`a` — выручка «Базовый» по неделям, `b` — «Расширенный», окна сдвинуты на две недели; `read_sql`, `payments`, `status = 'success'`; окружение курса.

Типы и память

решения, которые pandas принял за вас

Голосование: сколько весит `payments` в `pandas`?

3 147 409 строк, шесть колонок.

В базе таблица занимает **293 МБ** — без индексов.

Сколько займёт кадр после `pd.read_sql`?

50 МБ

250 МБ

**около
гигабайта**

Размер в базе — `pg_table_size`: это метаданные, таблица не читалась.

Ответ: 846,9 МБ — и больше половины съели два uuid

Колонка	Тип	МБ	Б/строку
payment_id	object	226,6	72
subscription_id	object	226,6	72
status	str	176,2	56,0
payment_method	str	167,1	53,1
paid_at	datetime64	25,2	8
amount	float64	25,2	8
Всего		846,9	269,1

- Два uuid — **453,2 МБ**, всё остальное — 393,6 МБ
- uuid приехал не строкой, а объектом uuid.UUID
- `df.info()` напишет `memory usage:` 807.6 МБ — то же число в МиБ

Выбирайте колонки в SQL. Для выручки оба uuid не нужны, а это больше половины веса кадра.

`memory_usage(deep=True)`, payments целиком через `read_sql`; окружение курса, pandas 3.0.5, без pyarrow; снято не в аудитории. `read_parquet` с pyarrow — 412,5 МБ (слайд 16): там и uuid — строки.

Деньги приехали не тем типом

В базе amount — numeric(12,2), точный до копейки. Тип в кадре `pd.read_sql` выбирает сам:

```
pd.read_sql(sql, engine)          float64    np.float64(199.0)
pd.read_sql(sql, engine, coerce_float=False) object    Decimal('199.00')
```

На `transactions.amount` (5 974 972 строк) сумма везде **8 047 584 511,17 Р**, но:

	<code>.sum()</code>	память колонки
float64	1,35 мс	47,8 МБ
Decimal	257,11 мс — в 191 раз дольше	669,2 МБ — в 14,0 раза больше

Ещё одна тихая: `.mean()` Decimal-колонки отдаёт `np.float64(1346.8823805651307)`.

Для агрегатов float64 достаточно: разница с базой **0,00 Р** на payments, transactions и cashback. Сверку до копейки делают **в базе**.

Окружение курса, pandas 3.0.5, без pyarrow; время — медиана из трёх прогонов.

Семь типов, которые вы встретите

Тип	Что хранит	Откуда у нас	Чем опасен
int64	целые без пропусков	region_code из read_sql	один пропуск — и молча float64
float64	дробные; пропуск — NaN	amount из read_sql	приближение: $0.1 + 0.2 == 0.3$ — False
str	текст; в pandas 3 — по умолчанию	status, payment_method	дорог: status — 56 байт на строку
datetime64[us, UTC]	момент времени с зоной	paid_at из read_sql	из CSV без parse_dates — str
category	коды + словарь значений	только astype("category")	на уникальных значениях — хуже
Int64	целые с пропуском <NA>	только astype("Int64")	сам не появится — надо попросить
object	любые объекты Python	uuid.UUID, Decimal, tariffs.valid_from	медленно: Decimal.sum() × 191

Тип выбирают источник и pandas, а не вы. Смотреть dtypes — **сразу после чтения**.

category: в пятьдесят раз легче — или хуже

Колонка	уникальных / строк	было	стало	итог
service_usage.device_type	4 / 7 968 633	435,6 МБ	8,0 МБ	легче в 54,7 раза
payments.payment_method	4 / 3 147 409	167,1 МБ	3,1 МБ	легче в 53,1 раза
clients.client_id	500 000 / 500 000	36,0 МБ	38,0 МБ	хуже на 5,6 %

category хранит каждое значение *один раз* в словаре, а в колонке — короткие коды. Четыре значения — словарь из четырёх строк и коды int8. 500 000 уникальных — словарь повторяет всю колонку, да ещё коды int32.

```
s = payments["payment_method"]
s.nunique() / len(s)
s = s.astype("category")
```

Правило: **доля уникальных** `nunique()` / `len()` мала — ориентир меньше 10 % — значит, category. У ключей она 100 %: будет только хуже.

`memory_usage(deep=True)` без индекса, таблицы целиком; окружение курса, pandas 3.0.5, без pyarrow.

Пропуск меняет тип

```
>>> subs['ended_at'].isna().sum()
167978
>>> subs['ended_at'].dt.year.dtype
float64
>>> subs['ended_at'].dt.year.value_counts(
...     dropna=False).sort_index()
ended_at
2024.0      20383
2025.0      62455
2026.0      83089
NaN         167978
Name: count, dtype: int64
>>> subs['ended_at'].dt.year.astype('Int64').dtype
Int64
```

- Пустой ended_at — штатный: подписка действует. Таких 167 978 из 333 905
- Пропуск не влезает в целое, и год стал float64 — отсюда **2024.0** с точкой
- astype("Int64") с заглавной: год — 2024, пропуск — <NA>
- У started_at пропусков нет, и .dt.year даёт int32

Сразу после чтения — три строки: df.dtypes, df.isna().sum(), df.memory_usage(deep=True).
Полный список — в конце лекции.

subscriptions целиком через read_sql; окружение курса, pandas 3.0.5, без pyarrow.

Пять способов прочитать — и один, который врёт

Как читаем	Типы	файл	чтение	в памяти
read_csv	не хранит: paid_at приезжает str	374,8 МБ	2,48 с	1 136,4 МБ
read_csv(parse_dates=...)	дату надо назвать самому	374,8 МБ	6,45 с	928,7 МБ
read_parquet*	хранит вместе с данными	251,0 МБ	0,069 с	412,5 МБ
read_excel	нужен openpyxl — у вас его нет			
read_json	тоже текст, типы угадываются			
read_sql	берёт из базы			846,9 МБ

payments целиком; окружение курса, pandas 3.0.5. * Снято с pyarrow — у вас его нет. Excel и JSON не замерялись.

Без движка чтение падает сразу:

```
clients = pd.read_parquet("clients.parquet")
ImportError: Unable to find a usable engine; tried using: 'pyarrow', 'fastparquet'.
report = pd.read_excel("report.xlsx")
ImportError: `Import openpyxl` failed. Use pip or conda to install the openpyxl package.
```

Упражнение до семинара — в main своего prime-monitor: `uv add pyarrow openpyxl`, затем `git diff` `uv.lock` и `git commit` — посмотрите, сколько пакетов приехало вместе с ними. Excel и Parquet сегодня — обзорно, практика — в теме 10.

Отбор

и запись, которая не доходит

Отбор: loc, iloc и маска

Что нужно	Как пишут
строки по условию	<code>payments[payments["status"] == "success"]</code>
строки и колонки	<code>payments.loc[payments["amount"] > 399, ["paid_at", "amount"]]</code>
по метке индекса	<code>failed.loc[3]</code>
по позиции	<code>payments.iloc[:5]</code>
несколько условий	<code>(p["status"] == "failed") & (p["amount"] > 199)</code>
значение из списка	<code>payments["payment_method"].isin(["card", "sbp"])</code>

- Маска — Series из True/False с тем же индексом, что у кадра. Условия — каждое в скобках, & вместо and
- Срез .loc включает правый конец, срез .iloc — нет

Читать можно как угодно. **Записывать** — только через .loc: почему — через два слайда.

b = a — не копия

```
fees = [199, 249, 399]
new_fees = fees
new_fees.append(449)
print(fees)
[199, 249, 399, 449]
```

- `new_fees = fees` не создаёт второй список. Это **второе имя** того же самого
- Поэтому 449 появилось и в `fees`: изменение дошло туда, куда его не звали
- Нужна копия — `list(fees)` или `fees.copy()`

Это было в ноутбуке **С2-идиомы**, раздел 6, — он лежит в репозитории курса на GitHub, там же задачи. Держите в голове эту интуицию: на следующем слайде pandas её перевернёт.

Copy-on-Write: что напечатает?

В чистом Python изменение дошло туда, куда не звали. Теперь pandas 3:

```
import pandas as pd
p = pd.DataFrame({"status": ["success", "failed", "success", "failed", "success"],
                  "amount": [399, 199, 449, 249, 399]})
failed = p[p["status"] == "failed"]
failed["amount"] = 0
print(p["amount"].sum())
```

1695

ничего не изменилось

1247

обнуление дошло до p

ошибка

pandas не дал так написать

Кадр собирается в самой ячейке: ни базы, ни сети. Запускаем вживую.

Ответ: 1695 — и ни одного предупреждения

Запись ушла в копию `failed`, а `p` не тронут. **В pandas 3 запись в срез не дойдёт — и никто не скажет.**

Как писать правильно — каждая форма на свежем `p`:

```
p.loc[p["status"] == "failed", "amount"] = 0 # p: 1247
p2 = p.assign(amount=p["amount"].mask(p["status"] == "failed", 0)) # p2: 1247, p: 1695
failed = p[p["status"] == "failed"].copy()
failed["amount"] = 0 # failed: 0, p: 1695
```

Однострочную цепочку pandas поймает:

```
p[p["status"] == "failed"]["amount"] = 0
```

```
<string>:4: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment.
Such chained assignment never works to update the original DataFrame or Series, because the intermediate object on which we are setting
values always behaves as a copy (due to Copy-on-Write).
Try using '.loc[row_indexer, col_indexer] = value' instead, to perform the assignment in a single step.
```

- Это **предупреждение**, а не исключение: код идёт дальше, сумма та же — 1695.
Двухстрочную форму не ловит ничего
- Обратная сторона: копия ленивая и почти бесплатная; правка в срезе или в результате метода не заденет исходный кадр, `inplace=` не нужен

Векторно против цикла: в секундах

Одна операция — `amount`, если `status == "success"`, иначе 0. Первые `N` строк `payments`:

Способ	10 000	100 000	3 147 409
векторно, <code>np.where(...)</code>	0,000437 с	0,0025 с	0,0664 с
<code>df["status"].apply(...)</code> по одной колонке	0,000858 с	0,0061 с	0,1846 с
<code>df.apply(..., axis=1)</code> — функция на строку	0,0230 с	0,2354 с	7,54 с
<code>for _, r in df.iterrows()</code>	0,0918 с	0,9308 с	29,9 с*

Мерить на своём объёме. На 10 000 строк `iterrows` проигрывает векторному 0,09 секунды — этого никто не заметит. На всей `payments` — полминуты на одну операцию.

Медиана из трёх; окружение курса, `pandas 3.0.5`, без `pyarrow`. * Пересчёт с 300 000 строк; прямой прогон — 30,2 с. На полном объёме медленнее векторного: `Series.apply` в 2,8 раза, `apply(axis=1)` в 113,5, `iterrows` в 450,6.

iterrows меняет типы

Числовой кадр: int64, int64, float64

```
numeric = pd.DataFrame({
    "region_code": [77, 50, 16],
    "birth_year": [1985, 1992, 2001],
    "amount": [399.0, 199.0, 449.0]})
for _, row in numeric.iterrows():
    break
type(row["region_code"])
print(row["region_code"])
```

```
<class 'numpy.float64'>
77.0
```

Код региона стал дробным: строка — это одна Series, и тип у неё один на всех.

Смешанный кадр: str, int64, float64

```
mixed = pd.DataFrame({
    "acquisition_channel": ["organic", "crm",
                           "social"],
    "region_code": [77, 50, 16],
    "amount": [399.0, 199.0, 449.0]})
for _, row in mixed.iterrows():
    break
print(row.dtype)
type(row["region_code"])
```

```
object
<class 'int'>
```

Значение цело, но строка — object: арифметика по ней идёт через объекты Python. Отсюда и полминуты.

Ни одна из этих ошибок не падает.

Код отработал, число напечаталось
и выглядит правдоподобно.

Прибор один — **число,**
которое вы знали до расчёта.

Группировка потеряла 10 000 клиентов

```
>>> clients.groupby('acquisition_channel')
...      ['client_id'].count()
acquisition_channel
crm                58631
organic            117952
partner_telecom    88193
performance        107976
referral           38952
social             78296
Name: client_id, dtype: int64
```

Сумма по группам — **490 000**.

A len(clients) — **500 000**.

```
>>> clients.groupby('acquisition_channel',
...                  dropna=False)['client_id'].count()
...
social                78296
NaN                   10000
```

Теперь сумма — **500 000**, а `isna().sum()` по каналу — 10 000.

`groupby` по умолчанию **выбрасывает пропуски из ключа** — и ничего об этом не пишет. Рядом с каждой группировкой — `dropna=False` или `isna().sum()`.

Что делать с пропусками — методология, её читает курс «Методы анализа данных». Наше дело — заметить, что они пропали, и не потерять знаменатель. `clients` целиком через `read_sql`; окружение курса, `pandas 3.0.5`.

Дубликаты: 2 665 896 или 3 142?

```
>>> payments.duplicated(subset=["subscription_id", "amount"]).sum()
2665896
>>> payments.duplicated(subset=["subscription_id", "amount"], keep=False).sum()
3113185
```

Та же подписка и сумма — **не дубль, а следующий месяц**. Оба числа не про двойные списания.

Двойное списание — та же пара, но *через минуты*. Ищем так:

```
d = payments.sort_values("paid_at")
d["prev"] = d.groupby(["subscription_id", "amount"])["paid_at"].shift()
dups = d[(d["paid_at"] - d["prev"]) < pd.Timedelta(minutes=20)]
len(dups)
3142
```

Порог 5 минут — 945. От 15 минут до 25 суток — **3 142**

Без двойных списаний
выручка — **879 859 959 ₽**

drop_duplicates по той же
паре без времени оставит
137 569 608 ₽

payments целиком, все статусы; выручка — status = 'success'; окружение курса, pandas 3.0.5.

Приёмка таблицы: семь вопросов до первого расчёта

Вопрос	Строка кода	Ответ по payments
1 Сколько строк — и сколько я ждал?	<code>len(payments)</code>	3 147 409 — как в базе
2 Какие типы выбрал за меня pandas?	<code>payments.dtypes</code>	uuid — object, amount — float64
3 Сколько таблица весит?	<code>payments.memory_usage(deep=True).sum()</code>	846,9 МБ
4 Одна строка — один кто? Нет ли дублей?	<code>payments["payment_id"].is_unique</code>	да; двойных списаний 3 142
5 Где пропуски?	<code>payments.isna().sum()</code>	amount, paid_at, status — 0
6 Нет ли значений, которых не может быть, и противоречий между колонками?	<code>payments["status"].value_counts()</code>	success, failed, refunded
7 Какой период покрыт и нет ли дыр?	<code>payments["paid_at"].dt.date.nunique()</code>	600 дней из 602: нет 14–15.04.2026

Вопрос 1 — **инвариант**: то, что обязано выполняться; здесь — число строк, известное заранее. Первый раздел семинарского ноутбука — **эти семь вопросов по вашей таблице из assignments**.

Что унести с этой пары

- **До расчёта назовите число, которое ждёте**

Ошибки pandas не падают — их видно только по расхождению с ним

- **Тип колонки выбрали за вас** — смотрите dtypes сразу после чтения

Деньги — float64, год с пропуском — float64, uuid — object

- **В кадр пишут через .loc[маска, колонка]**, а не в срез

- **groupby выбрасывает пропуски** — dropna=False или isna().sum() рядом

- **Колонки выбирают в SQL:** два uuid — больше половины веса payments

Семь вопросов приёмки задают любой таблице до того, как что-то по ней считать. С них начнётся семинар — уже по вашей таблице.

Что дальше сегодня

Сегодня

- **14:40 — ПР-1**, через двадцать минут. 10 минут, на бумаге.
Темы 01–02 и первый семинар.
Сегодняшние лекции не спрашиваются
- **14:50 — Л4**: соединения и агрегации
- **В конце Л4 — Проект 1**

К семинару

23.09 — среда (262) · 26.09 — суббота (261)

- Всё — в своём prime-monitor:
`uv sync`
- По желанию, в main и с коммитом:
`uv add pyarrow openpyxl`
- **ДЗ-1 — 27.09, 23:59**
приём со снижением балла до 04.10

Сегодня мы ни разу не соединили две таблицы: всё, что сломалось, сломалось внутри одной. **После проверочной работы добавим вторую.**